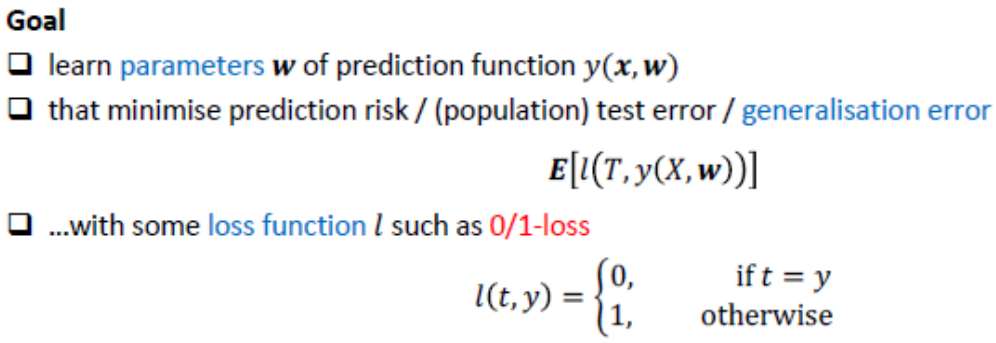
Week 3

Introduction to the Classification Problem

- **The Goal:** In classification, we are given a dataset $D = \{(x_1, t_1), ..., (x_N, t_N)\}$ where x represents our input features and t represents our discrete target labels (e.g., $c_1, \ldots, c_K$).
- **The Objective:** We want to learn the parameters w of a prediction function $y(x, w)$ that minimizes our prediction risk or generalization error.

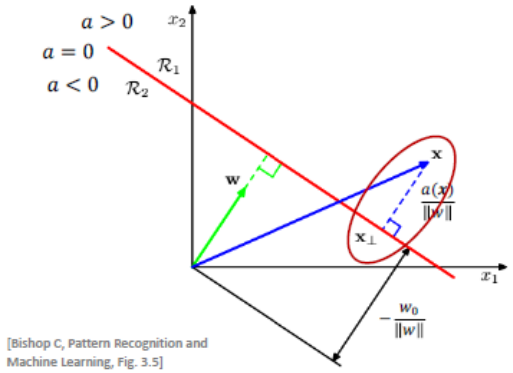


- **The Loss Function:** Ideally, we want to minimize the "0/1-loss", which simply gives a penalty of 1 if we guess wrong, and 0 if we guess right.

Insight: The 0/1 loss is what we *actually* care about (accuracy), but mathematically, it is a nightmare. It is a step function—flat almost everywhere with a sudden jump. You cannot calculate a gradient for a flat line! Because computers learn via calculus (gradients), we must replace the 0/1 loss with a smooth, differentiable "surrogate" loss function. That is exactly what Logistic Regression introduces.

The Geometric Perspective & Discriminative Models

Discriminative Models



[Bishop C, Pattern Recognition and Machine Learning, Fig. 3.5]

Decision boundary: $y(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} = 0$

If $y(\boldsymbol{x}_1) > y(\boldsymbol{x}_2) > 0$ then, relatively, more certain that $t_1 = 1$ than $t_2 = 1$.

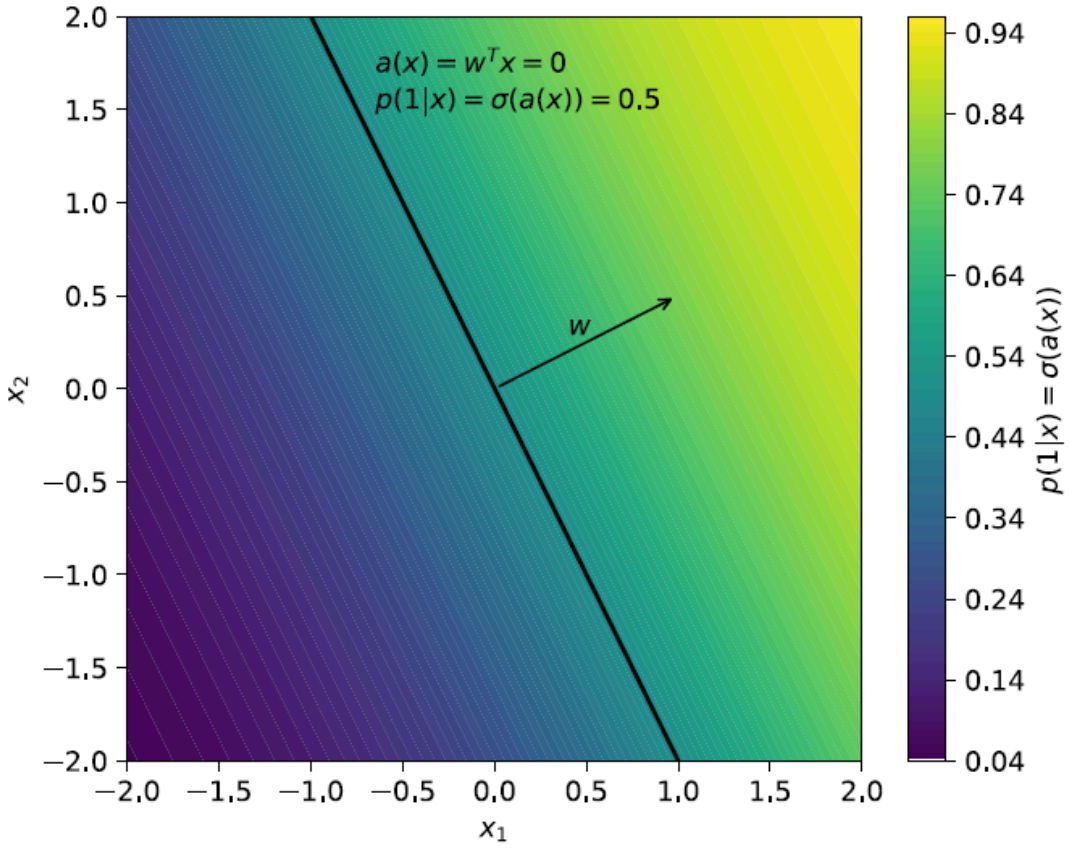
Problem:
What does $y(\boldsymbol{x}_1)$ mean in absolute terms?

Would like probabilistic model:

$$p(T = 1|\boldsymbol{X} = \boldsymbol{x})$$

- **Decision Boundary:** We start with a linear function $y(x) = w^T x$. The boundary where the model is "undecided" is where $w^T x = 0$.
- **The Problem:** If we just use a raw linear output, a larger value (like $y(x_1) > y(x_2) > 0$) makes us *relatively* more certain that x_1 belongs to class 1 than x_2 does. But what does a raw number like 42.5 or 0.001 mean in absolute terms?.
- **The Fix:** We want a **probabilistic model** that outputs exactly $p(T = 1|X = x)$—a number strictly between 0 and 1.
- which can tell us how probable is getting 1 instead of a raw number ranging from inf to -inf which is vague

Enter Logistic Regression: Logits and Sigmoids



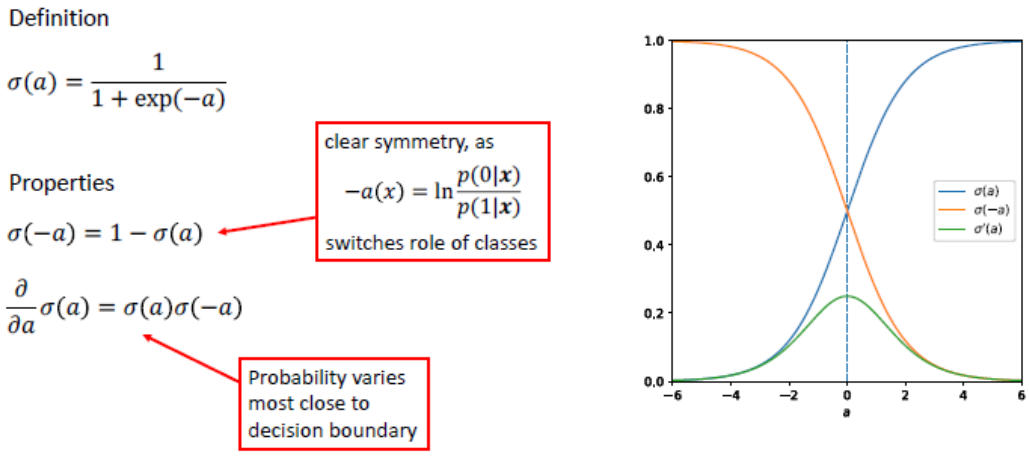
- **The Big Idea:** A linear function maps to the whole real line $(-\infty, \infty)$, so it cannot directly represent a probability $[0, 1]$.
- **Log Odds:** Instead of predicting probability directly, we let the linear function predict the **log odds** (also called the logit):
  - $a(x) = w^T x = \ln \frac{p(T=1|X=x)}{p(T=0|X=x)}$.
  - odd is just the ratio of an event happening over not happening
  - we use log odd instead of odds because odds are asymmetrical
    - odds can scale from 0 to inf
    - if likely to T=1 , can scale from 1 to inf
    - if likely to T=0, the odds is between 0 and 1 only
    - so log stretches this tiny range between 0 and 1 down to -inf and the large odds up to +inf

- **The Sigmoid Function:** To get our probability back (from the log odds which can range from -inf to inf) , we apply the inverse of the logit function, which is the **Sigmoid function**,
  - $\sigma(a) = \frac{1}{1+\exp(-a)}$.

$$\sigma(a) = \frac{1}{1+\exp(-a)}$$

$$= \frac{1}{1+\frac{p(0|\boldsymbol{x})}{p(1|\boldsymbol{x})}} = \frac{p(1|\boldsymbol{x})}{p(1|x)+p(0|\boldsymbol{x})} = p(1|\boldsymbol{x})$$

- **Properties:** The sigmoid beautifully squashes any real number into a probability between 0 and 1. It is also symmetric: $\sigma(-a) = 1 - \sigma(a)$.



Insight: Think of "odds" like betting odds at a horse race. If a horse has a 75% chance of winning, the odds are 3-to-1 (0.75 / 0.25 = 3). The *log* of those odds allows the scale to stretch from negative infinity to positive infinity. The Sigmoid function is simply the mathematical bridge taking us back from that infinite line into the comfortable probability zone of 0% to 100%.

Example: this S-curve mapping hours of study to the probability of passing an exam.

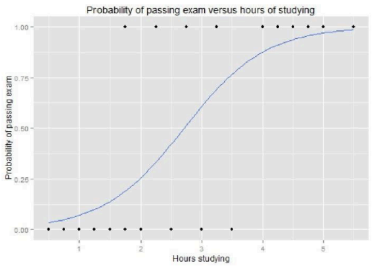
View as Generalised Linear Model

Predict probability / expected value of T:

$$y(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T\boldsymbol{x} + w_0) = \mathbf{E}[T|X = \boldsymbol{x}]$$

Example: probability of passing an exam ($T = 1$) versus hours of study (X)

Hours	0.50	0.75	1.00	1.25	1.50	1.75	1.75	2.00	2.25	2.50	2.75	3.00	3.25	3.50	4.00	4.25	4.50	4.75	5.00	5.50
Pass	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1



Hours of study	Probability of passing exam
1	0.07
2	0.26
3	0.61
4	0.87
5	0.97

$$y(x) = \sigma(w_1 x + w_0)$$
$$= \sigma(1.5x - 4.078)$$

Task: learn parameters $\boldsymbol{w}$

9

- the sigmoid graph is highly symmetrical
- the slope is steepest in the middle of the graph which is the decision boundary and this is where the prediction changes the most volatile
- every small change near the decision boundary results in big change in final prediction probability (refer the table)

Maximum Likelihood & The Logistic Loss

How do we actually train this to get the parameters? By using Maximum Likelihood Estimation (MLE).

- We want to maximize the probability of the data we actually observed.

Use maximum likelihood to learn model parameters $\boldsymbol{w}$ from iid data

$$D = \{(x_1, t_1), \ldots, (x_N, t_N)\}$$

With convention $t_n = 1$ (positive) and $t_n = 0$ (negative) and $y_n = p(1|\boldsymbol{x}_n) = \sigma(\boldsymbol{w}^T \boldsymbol{x}_n)$ can write negative log likelihood function as

$$\begin{aligned} -L(\boldsymbol{w}) &= -\ln \prod_{n=1}^{N} p(t_n|\boldsymbol{x}_n, \boldsymbol{w}) \\ &= -\ln \prod_{n=1}^{N} y_n^{t_n}(1-y_n)^{1-t_n} \\ &= -\sum_{n=1}^{N} t_n \ln y_n + (1-t_n)\ln(1-y_n) \\ &= \sum_{n=1}^{N} l_{\log}(t_n, y_n) \end{aligned}$$

tricky way to write case distinction $\begin{cases} y_n, & t_n = 1 \\ 1-y_n, & t_n = 0 \end{cases}$

Gives rise to logistic loss:

$$l_{\log}(t, y) = \begin{cases} -\ln y, & t = 1 \\ -\ln(1-y), & t = 0 \end{cases}$$

- If we use the convention where $t_n = 1$ is positive and $t_n = 0$ is negative, we can write the likelihood cleverly as: $y_n^{t_n}(1-y_n)^{1-t_n}$.
- where y_n is the predicted probability of data being 1 (between 0 and 1)
- Since the True value can only be either 1 or 0, notice that if $t_n = 1$, the right term becomes 1, and we just look at y_n. If $t_n = 0$, the left term becomes 1, and we look at $(1-y_n)$.
- the reason we use a negative log likelihood is because $y_n^{t_n}(1-y_n)^{1-t_n}$ is the multiple of values between 0 and 1 and this equates to a very small value close to 0 (computer rounds to 0), log changes this to Sum
- the negative is because we are finding the minimum of the curve to get the best weights for the model (more detail)
- **Negative Log-Likelihood:** Taking the negative natural logarithm gives us our loss function, widely known as the **Cross-Entropy Loss** or Logistic loss: $E(w) = -\sum(t_n \ln y_n + (1-t_n)\ln(1-y_n))$.

To minimise corresponding error function

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N} t_n \ln y_n + (1-t_n)\ln(1-y_n)$$

$$= -\sum_{n=1}^{N} t_n \ln \sigma(\boldsymbol{w}^T\boldsymbol{x}_n) + (1-t_n)\ln \sigma(-\boldsymbol{w}^T\boldsymbol{x}_n)$$

find its gradient

$$\nabla E(\boldsymbol{w}) = -\sum_{n=1}^{N} t_n \nabla \ln \sigma(\boldsymbol{w}^T\boldsymbol{x}_n) + (1-t_n)\nabla \ln \sigma(-\boldsymbol{w}^T\boldsymbol{x}_n)$$

$$= -\sum_{n=1}^{N} t_n(1-y_n)\boldsymbol{x}_n - (1-t_n)y_n\boldsymbol{x}_n$$

$$= -\sum_{n=1}^{N} (t_n - y_n)\,\boldsymbol{x}_n$$

Using chain rule and properties of σ:

$$\frac{\partial}{\partial w_i}\ln \sigma(\boldsymbol{w}^T\boldsymbol{x}_n)$$

$$= \frac{1}{\sigma(\boldsymbol{w}^T\boldsymbol{x}_n)}\frac{\partial}{\partial w_i}\sigma(\boldsymbol{w}^T\boldsymbol{x}_n)$$

$$= \frac{\sigma(\boldsymbol{w}^T\boldsymbol{x}_n)\sigma(-\boldsymbol{w}^T\boldsymbol{x}_n)}{\sigma(\boldsymbol{w}^T\boldsymbol{x}_n)}\frac{\partial}{\partial w_i}\boldsymbol{w}^T\boldsymbol{x}_n$$

$$= \sigma(-\boldsymbol{w}^T\boldsymbol{x}_n)x_{n,i}$$

$$\Downarrow$$

$$\nabla \ln \sigma(\boldsymbol{w}^T\boldsymbol{x}_n) = \sigma(-\boldsymbol{w}^T\boldsymbol{x}_n)\boldsymbol{x}_n$$

$$= (1-y_n)\boldsymbol{x}_n$$

- **The Gradient:** Taking the derivative of this loss function with respect to our weights w (using the chain rule) yields a remarkably elegant result: $\nabla E(w) = -\sum(t_n - y_n)x_n$.

Insight: Look closely at that gradient: $(t_n - y_n)x_n$. It literally means `(Actual_Label - Predicted_Probability) * Input_Features`. If your prediction exactly matches the target, the error is 0, and the gradient is 0 (no learning needed). If you are wrong, you adjust your weights exactly in proportion to *how wrong* you are and the magnitude of the input. So if the model predicted 0.8 and it is True (1), the error is 0.2 and we multiply that by the input to see how much to adjust. It is one of the most beautiful and intuitive results in machine learning!

Gradient Descent & Warning Signs

$$\nabla E(\boldsymbol{w}) = -\sum_{n=1}^{N} (t_n - y_n)\,\boldsymbol{x}_n$$

$$= \boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{t})$$

$$= \boldsymbol{X}^T(\sigma(\boldsymbol{X}\boldsymbol{w}) - \boldsymbol{t})$$

Similar to gradient in (least squares) linear regression:
$\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{t})$

"vectorised" form

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_N \end{bmatrix} \quad \boldsymbol{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix} \quad \boldsymbol{y} = \begin{bmatrix} \sigma(\boldsymbol{x}_1^T\boldsymbol{w}) \\ \sigma(\boldsymbol{x}_2^T\boldsymbol{w}) \\ \vdots \\ \sigma(\boldsymbol{x}_N^T\boldsymbol{w}) \end{bmatrix} = \sigma(\boldsymbol{X}\boldsymbol{w})$$

input matrix target vector prediction vector

Stochastic Gradient Update

$$\boldsymbol{w}_\tau = \boldsymbol{w}_{\tau-1} - \eta\tilde{\boldsymbol{X}}^T(\sigma(\tilde{\boldsymbol{X}}\boldsymbol{w}_{\tau-1}) - \tilde{\boldsymbol{t}})$$

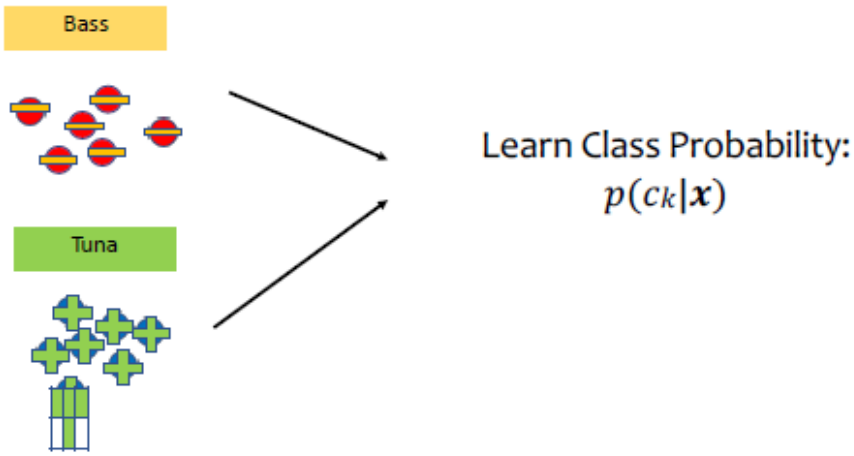
where $\tilde{\boldsymbol{X}}$ and $\tilde{\boldsymbol{t}}$ contain rows of $\boldsymbol{X}$ and $\boldsymbol{t}$ in batch of iteration τ

- **Vectorized Gradient:** In practice, we compute this for all data points simultaneously using matrices: $X^T(\sigma(Xw) - t)$. We use this gradient to update our weights iteratively: $w_\tau = w_{\tau-1} - \eta\nabla E$.
- **The Big Warning:** Gradient descent **does not converge** for linearly separable data! An example of this is that if all students who study above 3 hours pass (1) while all those who study below 3 hours fail (0).
- **Why?** If a dataset can be perfectly separated by a line, the model will try to push the predicted probability $p(t_n|x_n)$ exactly to 1.0 or exactly to 0.0 for every point. Looking at the sigmoid curve, the only way to get exactly 1.0 is if the input weight magnitude (w) goes to infinity.
- **The Solution:** We must use **regularization** (like L2 penalty) to constrain the weights and prevent them from blowing up.
- Generalisation to multiple classes possible (uses variant of sigmoid called “soft max”)

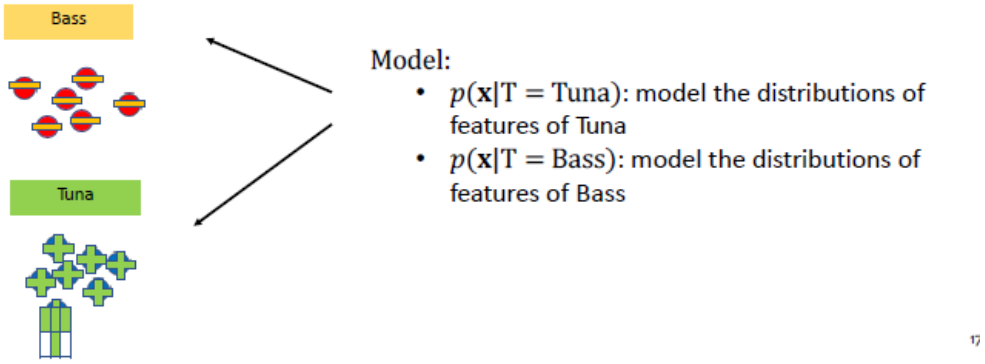
Discriminative vs. Generative Classifiers

First, we need to distinguish our two primary philosophies for classification:

- **Discriminative Models (e.g., Logistic Regression):** These try to learn $p(c_k|x)$ directly by searching for a decision boundary that separates the classes. It looks at a fish and asks, *"Is this side of the line Tuna or Bass?"*



- **Generative Models:** These try to **model each class's unique conditional distributions** $p(x|c_k)$. Like it is memorizing the recipe of creating data that statistically belongs to a class. In plain English, they ask, *"What does a typical Tuna look like, and what does a typical Bass look like?"* Because generative models understand the underlying distribution of the data, they can actually be used to *generate* new synthetic input data—just like DALL-E or Midjourney generate images from learned distributions of pixels from each term in the generation prompt (bear, 80s style etc)!



To make classifications, generative models rely on **Bayes' Theorem** to flip the probability around to what we actually want:

- $p(c_k|x) = \frac{p(x|c_k)p(c_k)}{p(x)}$

Then obtain required class probabilities via

$$p(c_k|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|c_k)p(c_k)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x}|c_k)p(c_k)}{\sum_{k=1}^{K} p(\boldsymbol{x}|c_k)p(c_k)}$$

Why?

- Model of full joint distribution allows for imputation of missing values, generation of alternative inputs (e.g., in AI art or "deep fakes").
- Class conditionals often straightforward to model/learn.

The Two Ingredients: Priors and Class Conditionals

To use Bayes' Theorem, we have to learn two distinct pieces of information from our training data:

Class Prior Distribution

The class prior distribution is a simple discrete categorical (or multinomial) distribution given by K probabilities

$$\boldsymbol{\varphi} = (\varphi_1, \ldots, \varphi_K)$$

that add up to 1:

$$\varphi_1 + \varphi_1 + \cdots + \varphi_K = 1$$

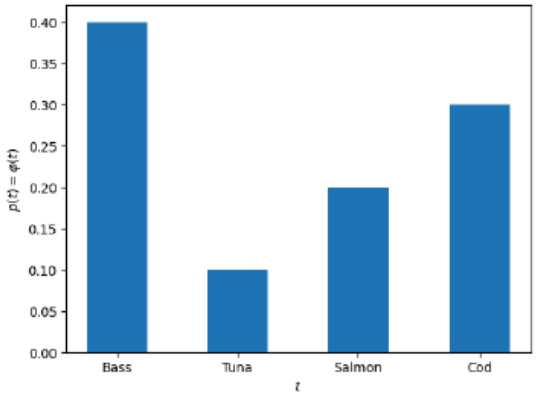
That is, the mass function is simply:

$$p(c_k|\boldsymbol{\varphi}) = \varphi_k$$

The maximum likelihood estimate is given by counting occurrence of classes:

$$\varphi_k = \frac{N_k}{N}$$

N_k ← number of data points of class k



1. The Class Prior Distribution $p(c_k)$: This is simply how common a class is in the real world before we even look at the features. If your dataset is 40% Bass and 10% Tuna, your prior probability of picking a Bass at random is 0.40. We estimate this by simply counting occurrences: $\varphi_k = \frac{N_k}{N}$. This is just the baseline probability without looking at any features of the classes yet.

Class Conditional Densities

For single continuous input variable X, can assume that normal class conditional:

$$p(x|c_k) = \frac{1}{\sqrt{2\pi}\sigma_k}\exp\left(-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right)$$

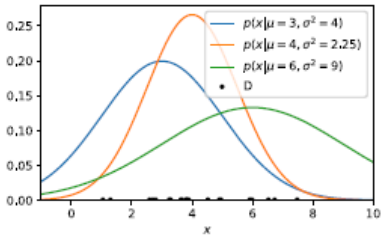
Need to fit parameters μ_k and σ_k for each class.

Split training data

$$D = \{(x_1, t_1), \ldots, (x_N, t_N)\}$$

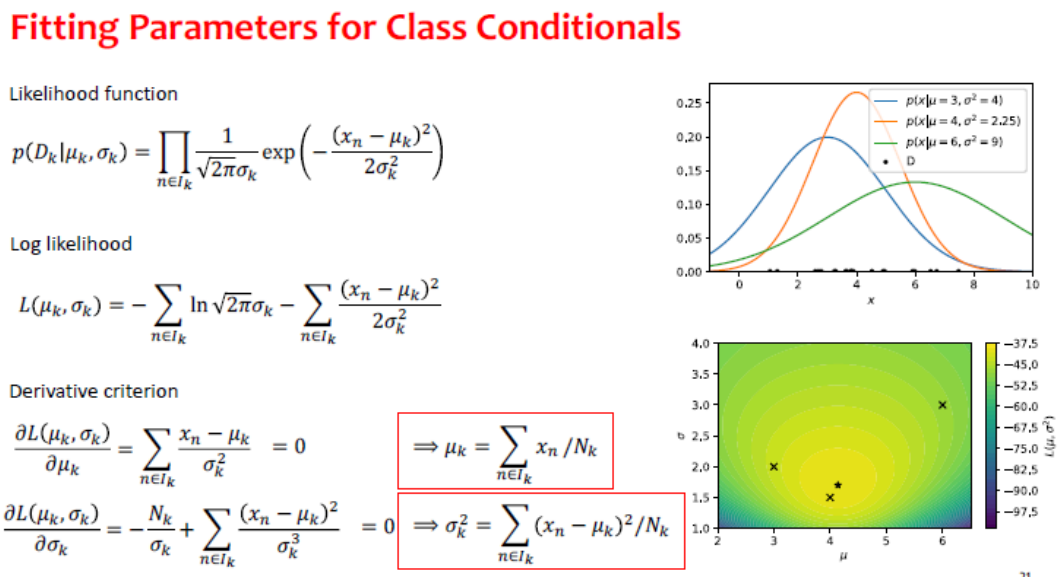
into K parts, one for each class

$$D_k = \{(x_n, t_n): n \in I_k\}$$
$$I_k = \{n: t_n = k\}$$



20

2. The Class Conditional Density $p(x|c_k)$: If we only have a single, continuous feature X (like fish length), we generally assume it follows a Normal (Gaussian) distribution. We fit a distinct bell curve for each class.
We find the best parameters using Maximum Likelihood, which wonderfully simplifies to calculating the standard sample mean (μ_k) and sample variance (σ_k^2) for the data points belonging to each specific class. For example we find the sample mean and sample variance for each of Tuna and Bass based on X.



Handling Multiple Features & Naïve Bayes

In real world scenarios, we do not usually only have 1 feature X, but multiple features or millions of pixels for an image where each pixel is a feature.

What happens when we have multiple features, like $X = (\text{length}, \text{width}, \text{weight})$? We have to model $p(x|c_k)$ in multiple dimensions.

Idea 1: The Naïve Bayes Assumption

The simplest approach is to assume that all features are conditionally independent of each other. Mathematically, you just multiply their individual 1D Gaussian probabilities together:
$p(x|c_k) = p(x_1|c_k)p(x_2|c_k)\ldots p(x_d|c_k)$.

- **Professor's Insight:** We call this "Naïve" because it is almost never true in reality. If a fish is longer, it is probably wider and heavier, too! Assuming

independence ignores these relationships.

Entering the Matrix: Covariance & Multivariate Normals

To fix the naive assumption, we use **Idea 2: The Multivariate Normal Distribution**. To do this, we must understand **Covariance**.

If you are calculating the variance of two things added together, you can't just add their variances. You have to account for how they move *together*:

Scenario: Mean and Variance of Grocery Stores Profits

Assume we have a grocery stores selling the following products:

Product	Profit	Sold	Mn. Sold $\mathbf{E}(X)$	Std. Sold $\sqrt{\mathbf{Var}(X)}$
apples	1	X_1	100	10
pears	2	X_2	50	10
cabbage	4	X_3	25	2

What is the expected value of the profit $Y = X_1 + 2X_2 + 4X_3$?

$$\begin{aligned}\mathbf{E}(Y) &= \mathbf{E}(X_1 + 2X_2 + 4X_3) \\ &= \mathbf{E}(X_1) + 2\mathbf{E}(X_2) + 4\mathbf{E}(X_3) \\ &= 100 + 100 + 100 = 300\end{aligned}$$

What is the variance / stan. dev. of the profit?

Can't tell without knowing covariances!!

- we know how each individual Product distributes and fluctuate (Std)
- For expected value of Y, we can just sum
- But for variance, we need to consider covariance because different RV (Products) might influence one another
- For example, people who buy apples are more likely to buy pears to make a fruit salad together, which is a positive interaction
- Covariance determines how 2 RV change together

Covariance

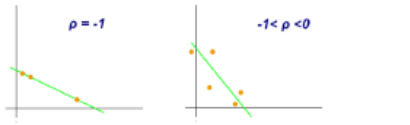
Covariance of two random variables X and Y with means μ and ν:

$$\mathbf{Cov}(X,Y) = \mathbf{E}\big((X-\mu)(Y-\nu)\big) = \mathbf{E}(XY) - \mathbf{E}(X)\mathbf{E}(Y)$$

determines variance of sums of random variables:

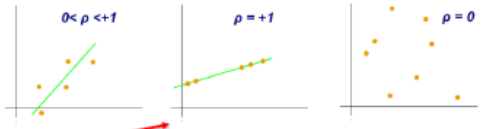
$$\mathbf{Var}(X+Y) = \mathbf{Var}(X) + \mathbf{Var}(Y) + 2\mathbf{Cov}(X,Y)$$

$$\rho(X,Y) = \frac{\mathbf{Cov}(X,Y)}{\sqrt{\mathbf{Var}(X)\mathbf{Var}(Y)}}$$



Properties:

- Generalises var.: $\mathbf{Var}(X) = \mathbf{Cov}(X,X)$
- Symmetric: $\mathbf{Cov}(X,Y) = \mathbf{Cov}(Y,X)$
- Shift invariant: $\mathbf{Cov}(X+c,Y+d) = \mathbf{Cov}(Y,X)$
- Bounded: $|\mathbf{Cov}(X,Y)| \leq \sqrt{\mathbf{Var}(X)\mathbf{Var}(Y)}$



gives rise to Pearson correlation coefficient (normalised covariance)

Covariance

Covariance of two random variables X and Y with means μ and ν:

$$\mathbf{Cov}(X,Y) = \mathbf{E}\big((X-\mu)(Y-\nu)\big) = \mathbf{E}(XY) - \mathbf{E}(X)\mathbf{E}(Y)$$

determines variance of sums of random variables:

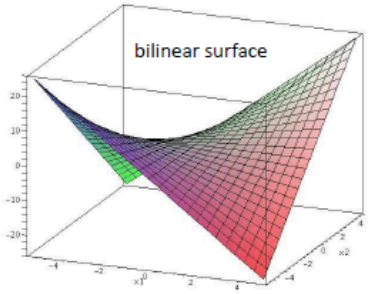
$$\mathbf{Var}(X+Y) = \mathbf{Var}(X) + \mathbf{Var}(Y) + 2\mathbf{Cov}(X,Y)$$

Bilinearity Proof

$$\begin{aligned}
&\mathbf{Cov}(aX+bY,Z)\\
&= \mathbf{E}\big((aX+bY)Z\big) - \mathbf{E}(aX+bY)\mathbf{E}(Z)\\
&= \mathbf{E}(aXZ+bYZ) - \big(\mathbf{E}(aX)+\mathbf{E}(bY)\big)\mathbf{E}(Z)\\
&= a\mathbf{E}(XZ) + b\mathbf{E}(YZ) - a\mathbf{E}(X)\mathbf{E}(Z) - b\mathbf{E}(Y)\mathbf{E}(Z)\\
&= a\mathbf{Cov}(X,Z) + b\mathbf{Cov}(Y,Z)
\end{aligned}$$

Properties:

- Generalises var.: $\mathbf{Var}(X) = \mathbf{Cov}(X,X)$
- Symmetric: $\mathbf{Cov}(X,Y) = \mathbf{Cov}(Y,X)$
- Shift invariant: $\mathbf{Cov}(X+c,Y+d) = \mathbf{Cov}(Y,X)$
- Bounded: $|\mathbf{Cov}(X,Y)| \leq \sqrt{\mathbf{Var}(X)\mathbf{Var}(Y)}$
- Bilinear: $\mathbf{Cov}(aX+bY,Z) = a\mathbf{Cov}(X,Z) + b\mathbf{Cov}(Y,Z)$



26

$$Var(X+Y) = Var(X) + Var(Y) + 2Cov(X,Y)$$

Covariance $Cov(X,Y) = E((X-\mu)(Y-\nu))$ tells us if two features are linked.

To manage all these relationships in multiple dimensions, we pack them into a $d \times d$ **Covariance Matrix** denoted by Σ. The diagonal holds the standard variance for each feature, and everywhere else holds the covariance between each pairs of features. So the covariance matrix maps all possible interactions

Scenario: Mean and Variance of Grocery Stores Profits

Assume we have a grocery stores selling the following products:

Product	Profit	Sold	Mn. Sold $\mathbf{E}(X)$	Std. Sold $\sqrt{\mathbf{Var}(X)}$
apples	1	X_1	100	10
pears	2	X_2	50	10
cabbage	4	X_3	25	2

Cov	X_1	X_2	X_3
X_1	100	50	-10
X_2	50	100	-10
X_3	-10	-10	4

ρ	X_1	X_2	X_3
X_1	1	0.5	-0.5
X_2	0.5	1	-0.5
X_3	-0.5	-0.5	1

What is the variance of the profit $Y = X_1 + 2X_2 + 4X_3$?

$$\begin{aligned}\mathbf{Var}(Y_1) = \mathbf{Var}(X_1 + 2X_2 + 4X_3) &= \mathbf{Cov}(X_1 + 2X_2 + 4X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1 + 2X_2 + 4X_3) + \mathbf{Cov}(2X_2 + 4X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1) + \mathbf{Cov}(X_1, 2X_2 + 4X_3) + \mathbf{Cov}(2X_2 + 4X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1) + 2\mathbf{Cov}(X_1, X_2) + 4\mathbf{Cov}(X_1, X_3) + \mathbf{Cov}(2X_2 + 4X_3, X_1 + 2X_2 + 4X_3)\end{aligned}$$

...

$$\begin{aligned}&= \mathbf{Cov}(X_1, X_1) + 2\mathbf{Cov}(X_1, X_2) + 4\mathbf{Cov}(X_1, X_3) + \mathbf{Cov}(2X_2 + 4X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1) + 2\mathbf{Cov}(X_1, X_2) + 4\mathbf{Cov}(X_1, X_3) + 2\mathbf{Cov}(X_2, X_1 + 2X_2 + 4X_3) + 4\mathbf{Cov}(X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1) + 2\mathbf{Cov}(X_1, X_2) + 4\mathbf{Cov}(X_1, X_3) + 2\mathbf{Cov}(X_2, X_1) + \mathbf{Cov}(2X_2 + 4X_3) + 4\mathbf{Cov}(X_3, X_1 + 2X_2 + 4X_3)\\ &= \mathbf{Cov}(X_1, X_1) + 4\mathbf{Cov}(X_1, X_2) + 4\mathbf{Cov}(X_1, X_3) + \mathbf{Cov}(2X_2 + 4X_3) + 4\mathbf{Cov}(X_3, X_1 + 2X_2 + 4X_3)\end{aligned}$$

28

...

$$\begin{aligned}&= \mathbf{Cov}(X_1, X_1) + 4\mathbf{Cov}(X_1, X_2) + 8\mathbf{Cov}(X_1, X_3) + 4\mathbf{Cov}(X_2, X_2) + 16\mathbf{Cov}(X_2, X_3) + 16\mathbf{Cov}(X_3, X_3)\\ &= 100 + 4 \cdot 50 - 8 \cdot 10 + 4 \cdot 100 - 16 \cdot 10 + 16 \cdot 4 = 524\end{aligned}$$

That was tedious! What is the general pattern?

General case: covariance between linear combinations

Let $\boldsymbol{x} = (X_1, X_2, \ldots, X_d)$ be random vector with covariances $\mathbf{Cov}(X_i, X_j) = \sigma_{i,j}$ and $a \in \mathbb{R}^d$ and $b \in \mathbb{R}^d$ coefficient vectors. Then covariance between linear combinations $\boldsymbol{a}^T\boldsymbol{x}$ and $\boldsymbol{b}^T\boldsymbol{x}$ is

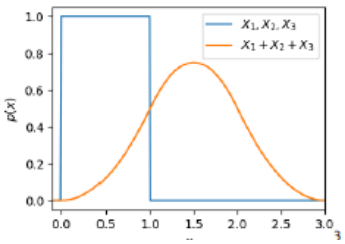
$$\begin{aligned}\mathbf{Cov}(\boldsymbol{a}^T\boldsymbol{x}, \boldsymbol{b}^T\boldsymbol{x}) &= \sum_{i=1}^{d}\sum_{j=1}^{d} a_i b_j \sigma_{i,j}\\ &= \sum_{i=1}^{d} a_i b_i \sigma_i^2 + 2\sum_{i=1}^{d}\sum_{j=i+1}^{d} a_i b_j \sigma_{i,j}\\ &= \boldsymbol{a}^T\boldsymbol{\Sigma}\boldsymbol{b}\end{aligned}$$

$\sigma_i^2 = \sigma_{i,i} = \mathbf{Var}(X_i)$

Special case: $\mathbf{Var}(\boldsymbol{a}^T\boldsymbol{x}) = \boldsymbol{a}^T\boldsymbol{\Sigma}\boldsymbol{a}$

The covariance matrix of $\boldsymbol{x}$ is $d \times d$-matrix $\boldsymbol{\Sigma}$ with entries $\boldsymbol{\Sigma}_{i,j} = \sigma_{i,j}$.

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1^2 & \sigma_{2,1} & \cdots & \sigma_{d,1} \\ \sigma_{1,2} & \sigma_2^2 & & \sigma_{d,2} \\ \vdots & & \ddots & \vdots \\ \sigma_{1,d} & \sigma_{2,d} & \cdots & \sigma_d^2 \end{bmatrix}$$



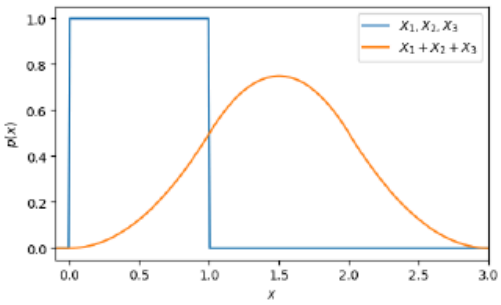
Shape of Linear Combinations of Random Variables

So we can easily compute variance and mean of sums.

What about shape (distribution family)?

In general: shape can change quite rapidly.

Exception: normal distribution!



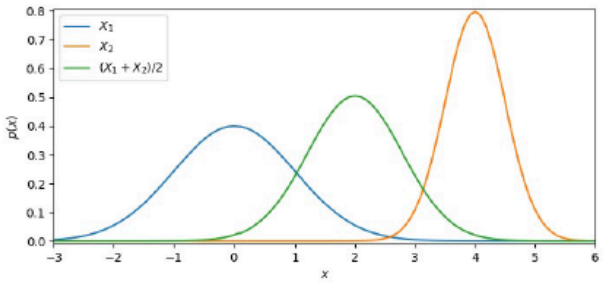
Example: sum of three independent uniform random variables

Linear Combinations of Normal Random Variables

The sum of independent and normally distributed $X_1 \sim \mathrm{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathrm{N}(\mu_2, \sigma_2^2)$ is also normal, i.e.,

$$X_1 + X_2 \sim \mathrm{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

More general: random vector of $\boldsymbol{x} = (X_1, \ldots, X_n)$ of independent normal $X_i \sim \mathrm{N}(\mu_i, \sigma_i^2)$ and $\boldsymbol{a} \in \mathbb{R}^n$ has normally distributed linear combination: $\boldsymbol{a}^T\boldsymbol{x} \sim \mathrm{N}(a^T\mu, a_1^2\sigma_1^2 + \cdots + a_n^2\sigma_n^2)$



What can we say about linear combinations of dependent normal?

32

Dependent Normal Random Variables

Are sums of dependent normal random variables always normal?

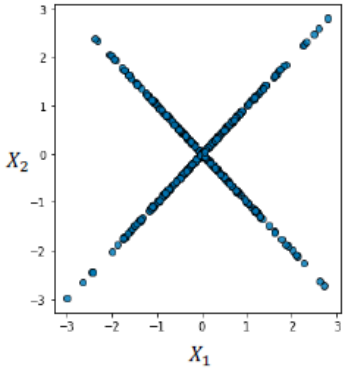
No!

Example: Consider game where, the first player receives/pays X_1 dollars where $X_1 \in \mathrm{N}(0,1)$. Then depending on outcome of coin flip, the second player receives/pays

$$X_2 = \begin{cases} X_1 & \text{if heads} \\ -X_1 & \text{otherwise} \end{cases}$$

With this, marginally, $X_2 \sim \mathrm{N}(0,1)$, but $Z = X_1 + X_2$ is not even a continuous random variable, because $\mathrm{P}(Z = 0) = 0.5$.

Demanding that all linear combinations of random vector are normal gives rise to multivariate normal distribution!



Now, we upgrade our 1D Gaussian formula to the full **Multivariate Normal Density** formula:

$$p(x) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)\right)$$

Multivariate Normal Distribution

Random vector $\boldsymbol{x} = (X_1, \ldots, X_d)$ with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$ follows multivariate normal distribution if for all coefficient vectors the linear combination is normal:

$$\boldsymbol{a}^T \boldsymbol{x} \sim \mathrm{N}(\boldsymbol{a}^T \boldsymbol{\mu}, \boldsymbol{a}^T \boldsymbol{\Sigma} \boldsymbol{a})$$

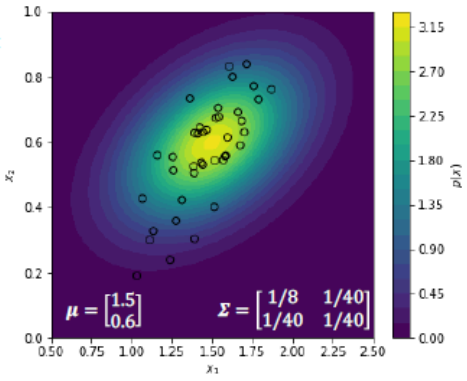
This is denoted by $\boldsymbol{x} \sim \mathrm{N}_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

The joint probability density function is given by:

$$p(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu})\right)$$

determinant (deformation of unit hypercube by matrix)

squared length of standardised version of value



generalises univariate case $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right)$

34

- **Professor's Insight:** Don't let this formula intimidate you! Notice how $(x-\mu)^2/\sigma^2$ from the 1D case just becomes $(x-\mu)^T\Sigma^{-1}(x-\mu)$ in linear algebra. It's doing the exact same thing: measuring how far a data point x is from the center μ, scaled by the spread of the data Σ.

This covers the foundational theory of Generative Models and the Covariance Matrix! Before we tackle the final part—where we actually fit these massive matrices to data and see how they create decision boundaries—does the difference between the Naïve Bayes assumption and a full Multivariate Normal distribution make complete sense?

Fitting the Full Multivariate Model & The Overfitting Danger

Assume again multiple (continuous) input variables: $\boldsymbol{X} = (X_1, \ldots, X_d) \in \mathbb{R}^d$.

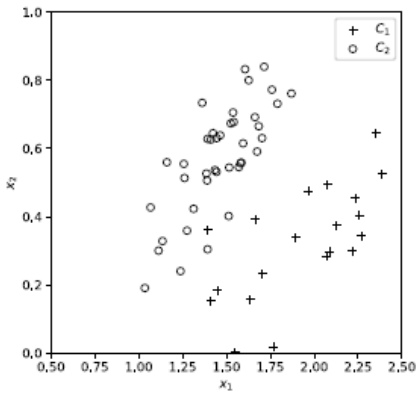
How to model class conditional input densities?

Idea 1: assume conditionally independent inputs

$$p(\boldsymbol{x}|c_k) = p(x_1|c_k)p(x_2|c_k)\cdots p(x_d|c_k)$$

-> allows to re-use previous univariate Gaussians
-> approach is called Naïve Bayes
-> however, independence assumption often violated

Idea 2: model as multivariate normal distribution



If we want to use a full Multivariate Normal distribution for each class, we have to calculate two main things from our training data for *every single class*:

1. **The Mean Vector (μ_k):** The average values of all features for class k.
2. **The Covariance Matrix (Σ_k):** How all the features vary and correlate with each other for class k.

Insight: The math to find these is thankfully straightforward—we just take the sample means and sample covariances of our data points. However, we hit a massive practical problem. If we have K classes and d features, we have to estimate $K \times d^2$ covariance parameters. If you have 100 features, that is 10,000 parameters *per class*! If your dataset is not absolutely huge, you run a severe risk of overfitting your data.

To fix this parameter explosion, we have two main workarounds:

Workaround 1: Diagonal Covariances

Parameter Fitting for Multivariate Normal Class Conditionals

Per class need to fit mean vector $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}_k$

$$p(\boldsymbol{x}|c_k) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}_k|}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k)\right)$$

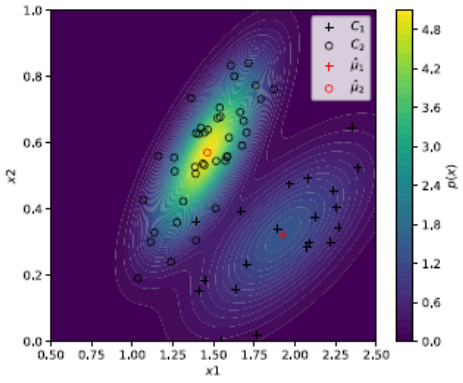
Split again data via classes and find ML estimates as

$$\boldsymbol{\mu}_k = \frac{1}{N_k}\sum_{n\in D_k} \boldsymbol{x}_n$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_{n\in D_k} (\boldsymbol{x}_n - \boldsymbol{\mu}_k)(\boldsymbol{x}_n - \boldsymbol{\mu}_k)^T$$

or component-wise

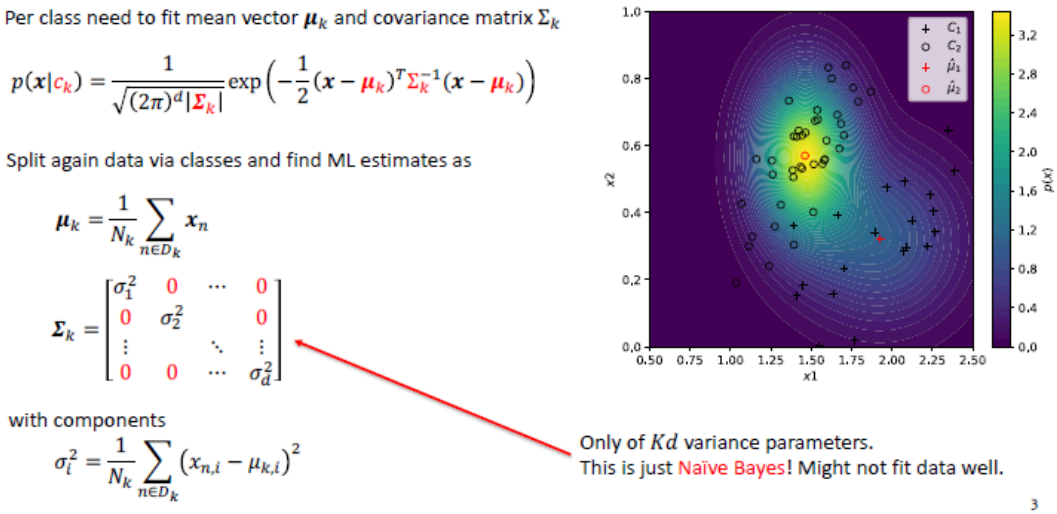
$$\sigma_{i,j} = \frac{1}{N_k}\sum_{n\in D_k}(x_{n,i} - \mu_{k,i})(x_{n,j} - \mu_{k,j})$$



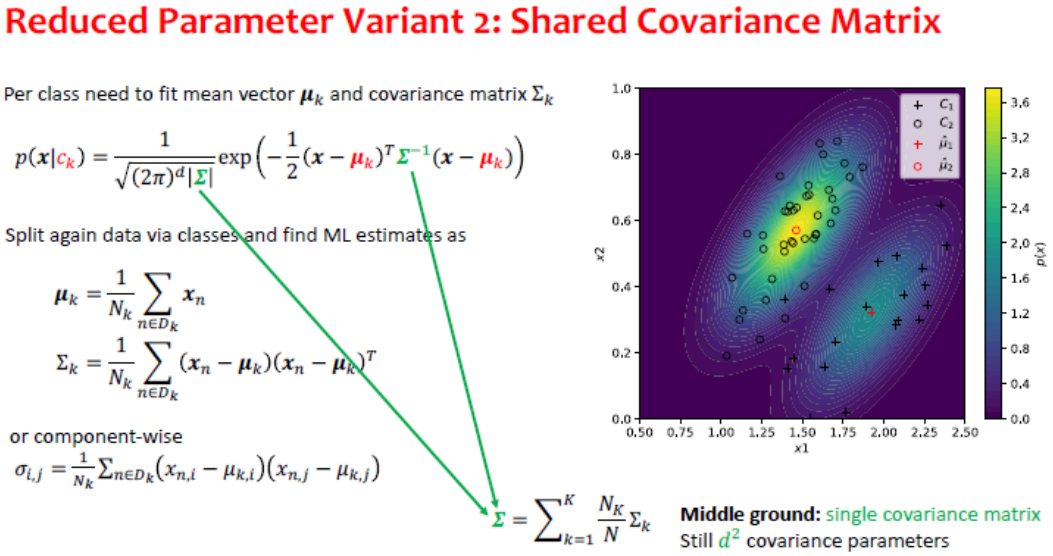
Total of Kd^2 covariance parameters to fit!
Danger of overfitting if $n \approx Kd^2$.

What if we just force all the non-diagonal elements of our covariance matrix to be exactly zero?

- This drastically reduces our parameter count from Kd^2 to just Kd.
- However, by doing this, we are telling the model to completely ignore feature correlations.
- **The Catch:** This is mathematically identical to making the **Naïve Bayes assumption** we talked about earlier! Because it assumes features act completely independently, it might not fit the real-world data very well but sometimes still work well in practice, surprisingly.



Workaround 2: Shared Covariance Matrix



This is the "Goldilocks" solution (often called Linear Discriminant Analysis or LDA). Instead of giving every class its own unique covariance matrix, we assume that all classes share the *exact same* covariance matrix, Σ.

- We estimate this shared matrix by taking a weighted average of the individual class matrices. So each individual class will center at different parts of the

graph based on their respective means but the shape of the covariance matrix is forced to be the same.

- **The Benefit:** We keep the ability to model feature correlations, but we only have to estimate d^2 covariance parameters total (instead of $K \times d^2$). This is a fantastic middle ground to prevent overfitting while retaining model power.

The Geometry of Decision Boundaries

Decision Boundary

Can use model for prediction with uncertainties, but can we describe decision boundary algebraically as before?

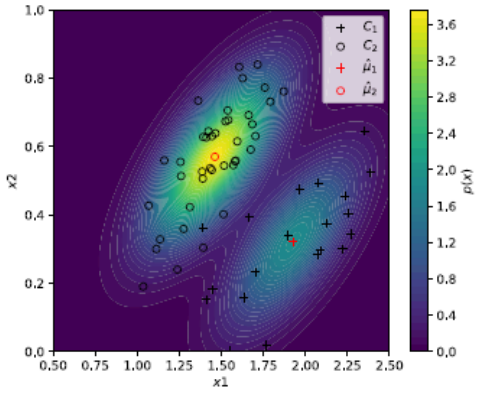
$$p(x|c_1)p(c_1) = p(x|c_2)p(c_2)$$

Alternatively: consider log odds.

$$a = \ln \frac{p(x|c_1)p(c_1)}{p(x|c_2)p(c_2)} = 0$$

$$= \ln \frac{\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_1)^T\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_1)\right)}{\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_2)^T\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_2)\right)} + \ln \frac{p(c_1)}{p(c_2)}$$

Is this still a linear model in the previous sense?



Now we reach the most visually satisfying part of the math! What does the boundary line between our classes actually look like? We find this by looking at the log odds: $a = \ln \frac{p(x|c_1)p(c_1)}{p(x|c_2)p(c_2)}$.

$$a = \ln \frac{\exp\left(-\frac{1}{2}(x-\mu_1)^T\Sigma^{-1}(x-\mu_1)\right)}{\exp\left(-\frac{1}{2}(x-\mu_2)^T\Sigma^{-1}(x-\mu_2)\right)} + \ln\frac{p(c_1)}{p(c_2)}$$

$$= \frac{1}{2}(x-\mu_2)^T\Sigma^{-1}(x-\mu_2) - \frac{1}{2}(x-\mu_1)^T\Sigma^{-1}(x-\mu_1) + \ln\frac{p(c_1)}{p(c_2)}$$

quadratic terms cancel because of assumption of single covariance matrix

$$= \frac{1}{2}x^T\Sigma^{-1}x - x^T\Sigma^{-1}\mu_2 + \frac{1}{2}\mu_2\Sigma^{-1}\mu_2 - \frac{1}{2}x^T\Sigma^{-1}x + x^T\Sigma^{-1}\mu_1 - \frac{1}{2}\mu_1\Sigma^{-1}\mu_1 + \ln\frac{p(c_1)}{p(c_2)}$$

$$= (\mu_1-\mu_2)^T\Sigma^{-1}x + \frac{1}{2}\mu_2^T\Sigma^{-1}\mu_2 - \frac{1}{2}\mu_1^T\Sigma^{-1}\mu_1 + \ln\frac{p(c_1)}{p(c_2)}$$

$a = w^Tx + w_0$ log odds again described by linear function of weights:

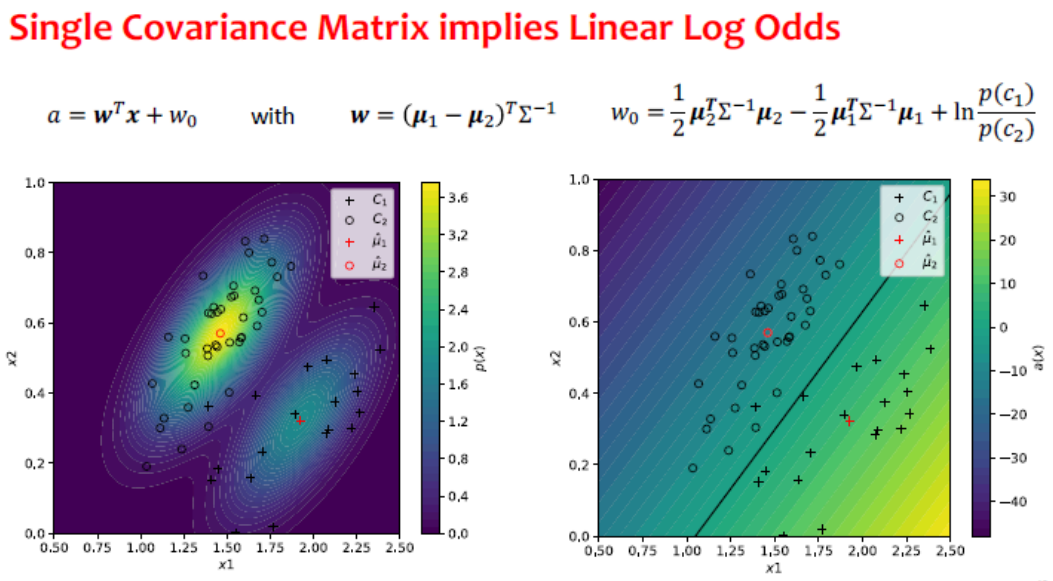
$$w = (\mu_1-\mu_2)^T\Sigma^{-1}$$

$$w_0 = \frac{1}{2}\mu_2^T\Sigma^{-1}\mu_2 - \frac{1}{2}\mu_1^T\Sigma^{-1}\mu_1 + \ln\frac{p(c_1)}{p(c_2)}$$

28

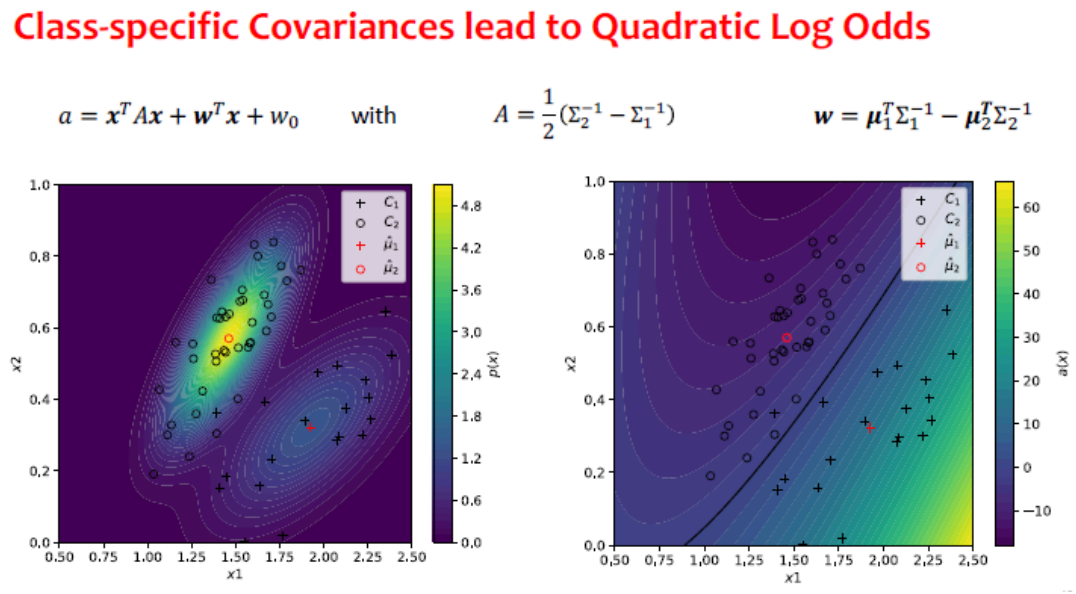
Linear Boundaries (Shared Covariance)

If we use the Shared Covariance matrix (Workaround 2), a mathematical miracle happens. When we expand the log odds equation, the quadratic terms (the complex x^2 parts) perfectly cancel each other out!



- We are left with a purely linear equation: $a = w^Tx + w_0$.
- Visually, this means the boundary separating our classes is a perfectly straight line (or a flat plane in 3D).

Quadratic Boundaries (Class-Specific Covariances) If we use the Full Multivariate model where every class gets its own unique covariance matrix, those x^2 terms do *not* cancel out.



- Our log odds formula retains an x^T A x term.
- Visually, our decision boundary is no longer a straight line; it curves into a quadratic shape (like an ellipse, parabola, or hyperbola), where the boundary smoothly wraps around the classes. This is often called Quadratic Discriminant Analysis (QDA).

Concluding Remarks

To wrap up this module, let's summarize the pros and cons of these Generative Models:

- **The Good:** Unlike Logistic Regression, which requires slow, expensive, iterative algorithms like Gradient Descent to find its parameters, Generative models with normal class conditionals can calculate their optimal parameters instantly using closed-form mathematical formulas.
- **The Bad:** they require to choose conditional input distributions (and sometimes multivariate normal might not capture the data well). You are strictly assuming your data actually looks like a bell curve (Normal

distribution). If your real-world data looks completely different, this model will struggle.

- **The Ugly (Parameters):** If you don't use Naïve Bayes or a Shared Covariance matrix, the number of parameters grows quadratically with your features, which is highly computationally expensive. Can use shared covariance to reduce parameters by factor K

Finally, while we focused heavily on continuous numbers here, remember that these models can easily be extended to handle discrete/categorical features, usually by applying the Naïve Bayes assumption to them.

The whole Generative Model architecture relies upon a single mathematical assumption that random variables should follow a normal distribution.